



Projet Commun Algo & ACL

Cahier des Charges : Application de Livraison

MORLET Hugo, CASTRO Alexis, SABATIER Guillaume, RAHALI Linda, Bensaadi Naël

11 janvier 2026

Table des matières

1	Description de l'entreprise et de ses besoins	3
2	Description du problème et objectifs du logiciel	3
2.1	Diagramme de Cas d'utilisation	4
2.2	Description détaillée des Cas d'Utilisation	4
2.2.1	Cas d'utilisation : Gérer les données d'entrée	4
2.2.2	Cas d'utilisation : Configurer la mission	5
2.2.3	Cas d'utilisation : Calculer la tournée	6
2.2.4	Cas d'utilisation : Visualiser la tournée	6
2.3	Diagramme de Séquences : Visualisation Graphique	7
2.4	Diagramme d'États-Transitions	7
3	Environnement et modèles	8
3.1	Méthodologie de gestion de projet	8
3.2	Diagramme de Classes	9
4	Exigences fonctionnelles (EF)	13
4.1	Identification des Acteurs	13
4.2	Liste des Exigences Fonctionnelles	13
5	Exigences non-fonctionnelles (ENF)	13
6	Définition des critères de qualité	14
6.1	CMM	14
6.2	Détermination des standards de qualité	14
6.2.1	Convivialité	14
6.2.2	Efficacité	15
6.2.3	La Conception Modulaire (Design Patterns)	15
6.2.4	Stratégie de Test et Fiabilisation	16
6.2.5	Facilité de maintenance	16
6.2.6	Réutilisabilité	16
7	Planification des étapes du développement	17
7.1	Jalons principaux	17
7.2	Diagramme de Gantt	18
8	Description des exigences d'assistance technique	19
9	Conclusion	19

1 Description de l'entreprise et de ses besoins

Au quotidien, une entreprise de transport doit gérer un flux de livraisons vers de nombreuses destinations en France. C'est un défi logistique qui repose sur deux piliers : d'une part, répartir les villes entre les différents camions et, d'autre part, définir l'ordre de passage pour chaque chauffeur. Jusqu'ici, ce sont les responsables d'exploitation qui s'en chargent manuellement. Malheureusement, cette méthode artisanale atteint rapidement ses limites. C'est un travail difficile et répétitif. De plus, les itinéraires définis ne sont pas toujours les plus courts à cause des erreurs humaines. Ainsi l'organisation manque parfois de fluidité et de précision.

Ce mode de fonctionnement actuel ne permet pas d'atteindre les objectifs de rentabilité fixés par l'entreprise. L'imprécision des tournées peut générer une frustration chez les Transporteurs.

Afin de pallier ces difficultés, ce projet propose le développement d'une solution logicielle. Celle-ci sera dédiée à l'automatisation de la logistique du problème de tournée de véhicules. En automatisant la répartition des destinations et le calcul d'itinéraires, cet outil vise à garantir une organisation optimale tout en réduisant considérablement le temps de préparation. Ce gain de temps et de qualité de la solution se traduira par une meilleure productivité et une rentabilité accrue pour la structure.

La compréhension complète de ce contexte, ainsi que des attentes opérationnelles, nous permet désormais d'établir les spécifications techniques du projet. Celles-ci sont détaillées dans les sections suivantes afin de garantir une réponse logicielle parfaitement adaptée aux réalités du terrain.

2 Description du problème et objectifs du logiciel

L'enjeu central de ce projet est de transformer la gestion des flux de transport en une opération automatisée et performante. Plutôt que de subir la complexité de la répartition des charges, le logiciel vise à orchestrer les tournées. Cela permet de minimiser les coûts opérationnels. La solution repose sur un moteur de calcul capable d'attribuer intelligemment les points de livraison aux camions disponibles, tout en garantissant un retour systématique au dépôt. Pour que cet outil soit réellement adopté sur le terrain, il faut la mise en place d'une interface utilisateur intuitive : la visualisation claire des trajets et une configuration simplifiée des paramètres de tournée sont au cœur de la conception. Sur le plan technique, ce défi nécessite une modélisation rigoureuse. Le projet s'appuie notamment sur des algorithmes, tels que ceux dédiés au problème du voyageur de commerce (VRP), afin de traiter les contraintes logistiques avec précision et d'assurer une exécution logicielle sans faille.

Cette section présente les diagrammes UML modélisant le système (Cas d'utilisation, Séquences, États-Transitions).

2.1 Diagramme de Cas d'utilisation

Le diagramme suivant illustre les fonctionnalités offertes à l'Opérateur. Les étapes sont détaillées ci-après.

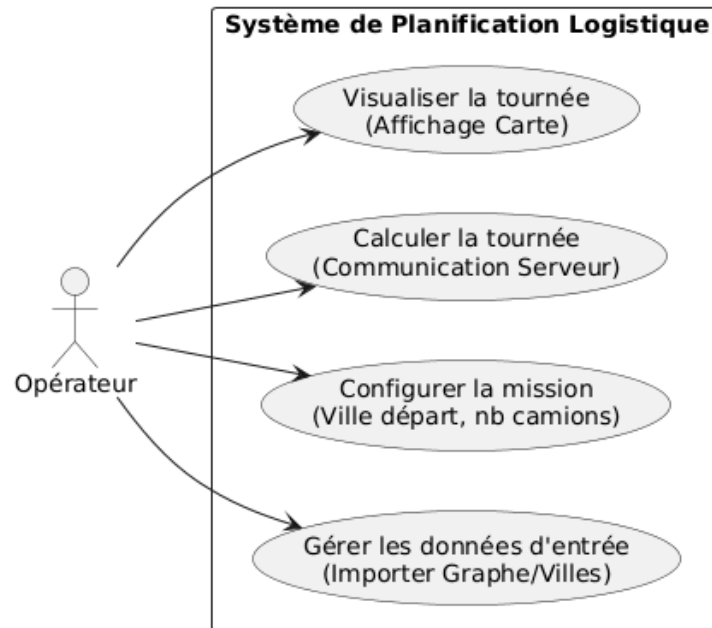


FIGURE 1 – Diagramme de Cas d'Utilisation de l'Application de Livraison

2.2 Description détaillée des Cas d'Utilisation

Voici la description textuelle des étapes pour les fonctionnalités principales.

2.2.1 Cas d'utilisation : Gérer les données d'entrée

Acteur : Opérateur

But : Importer le graphe routier (liste des villes) pour préparer les calculs.

Description : L'opérateur importe un fichier contenant les villes et leurs coordonnées pour construire le graphe.

Déroulement :

Action de l'Acteur	Réaction du Système
1. L'opérateur clique sur le bouton "Importer fichier".	2. Le système ouvre l'explorateur de fichiers.
3. L'opérateur sélectionne le fichier source (JSON/CSV).	4. Le système (Client) lit le fichier et vérifie son format.
	5. Le système confirme le chargement et affiche les villes sur la carte.

Post-conditions : Les villes sont chargées en mémoire et affichées sur la carte.

2.2.2 Cas d'utilisation : Configurer la mission

Acteur : Opérateur

But : Définir les contraintes de la livraison (dépôt de départ, taille de la flotte).

Pré-conditions : Des données (villes) sont chargées.

Description : L'opérateur paramètre la ville de départ et le nombre de camions disponibles.

Références : Gérer les données d'entrée (précède).

Déroulement :

Action de l'Acteur	Réaction du Système
1. L'opérateur sélectionne la ville de départ (Dépôt).	
2. L'opérateur saisit le nombre de camions (k).	3. Le système vérifie la validité de la saisie ($k > 0$).
4. L'opérateur valide la configuration.	5. Le système mémorise les paramètres pour le calcul.

Post-conditions : La mission est configurée et prête pour le calcul.

2.2.3 Cas d'utilisation : Calculer la tournée

Acteur : Opérateur

But : Obtenir les itinéraires optimaux (TSP) calculés par le serveur pour chaque camion.

Pré-conditions : Données chargées et mission configurée.

Description : Le client envoie les données au serveur C++ qui exécute l'algorithme TSP et retourne les tournées optimales. La visualisation reste un cas distinct déclenché par l'opérateur après le calcul.

Références : Configurer la mission (précède), Visualiser la tournée (optionnel, après calcul).

Déroulement :

Action de l'Acteur	Réaction du Système
1. L'opérateur clique sur "Lancer le calcul".	2. Le système vérifie les pré-conditions (données, config).
	3. Le système (Client) formate et envoie la requête au Serveur C++.
	4. Le système (Serveur) exécute l'algorithme TSP/VRP.
	5. Le système (Serveur) renvoie les résultats (listes de villes ordonnées).
	6. Le système (Client) notifie le succès ("Calcul terminé") et rend les résultats disponibles pour une visualisation ultérieure.

Post-conditions : Les tournées optimales sont calculées et stockées.

2.2.4 Cas d'utilisation : Visualiser la tournée

Acteur : Opérateur

But : Consulter graphiquement les trajets de chaque camion sur la carte.

Pré-conditions : Calcul terminé avec succès.

Description : L'opérateur visualise les tournées calculées avec une couleur distincte par véhicule.

Références : Calculer la tournée (précède).

Déroulement :

Action de l'Acteur	Réaction du Système
1. L'opérateur accède à l'écran de résultats.	2. Le système convertit les coordonnées (Lat/Long) en pixels.
	3. Le système trace les routes sur la carte.
	4. Le système applique une couleur distincte par camion pour différencier les tournées.

Post-conditions : Les tournées sont affichées graphiquement sur la carte.

2.3 Diagramme de Séquences : Visualisation Graphique

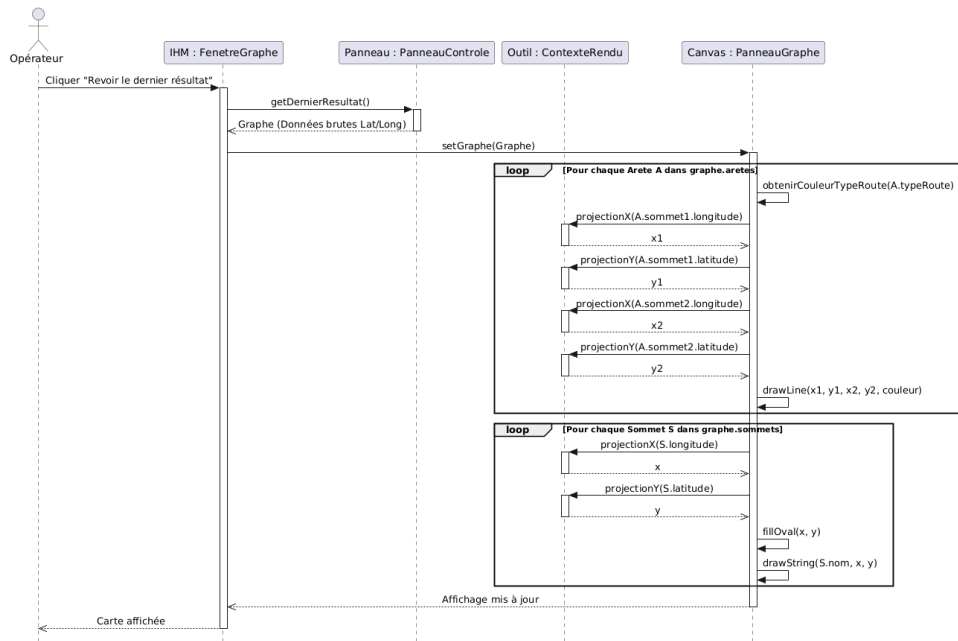


FIGURE 2 – Diagramme de séquences : Processus de dessin des tournées

2.4 Diagramme d'États-Transitions

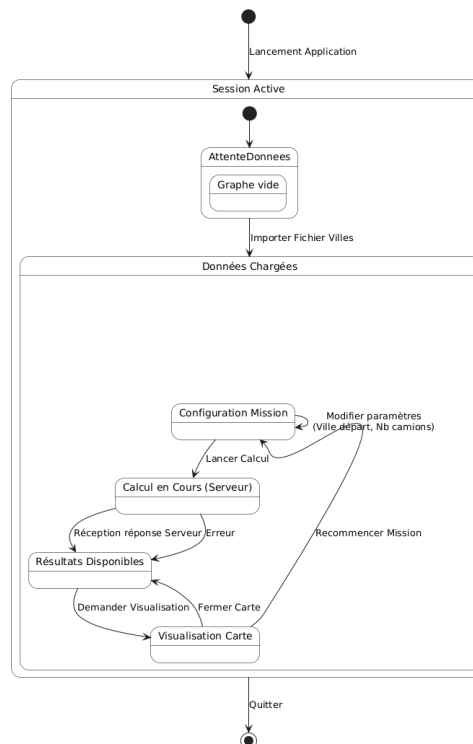


FIGURE 3 – Diagramme d'états-transitions de l'application cliente

3 Environnement et modèles

3.1 Méthodologie de gestion de projet

Durant ce projet, nous avons choisi d'adopter une démarche Agile. Ce choix s'est imposé comme la méthode la plus pertinente. Elle offre une solution adaptée aux défis techniques imposés par l'optimisation logistique. Un des principaux avantages est l'adaptabilité. En effet, cette méthode de travail nous a permis durant la totalité du projet, de pouvoir échanger simplement et de prendre des décisions régulièrement.

Cette flexibilité est d'autant plus importante que notre architecture est hybride. Elle mêle C++ pour le client et Java pour le serveur. Cette communication constante favorise une synchronisation continue entre ces deux pôles, réduisant ainsi les risques d'incompatibilité en fin de projet. En produisant des versions fonctionnelles de manière régulière, nous sécurisons l'avancement du logiciel. Cela permet également à l'interface de rester fidèle aux attentes réelles des utilisateurs de terrain.

Pour mettre en application cette méthode, nous avons mis en place un tableau Kanban. Cet outil, pilier de la méthode Agile, nous permet de visualiser en temps réel la répartition des tâches entre le développement du moteur algorithmique en Java et la conception de l'interface en C++, garantissant ainsi une meilleure fluidité dans notre flux de travail.

En définitive, ce cadre de travail s'est imposé comme l'approche la plus pertinente pour garantir la réussite de ce projet.

3.2 Diagramme de Classes

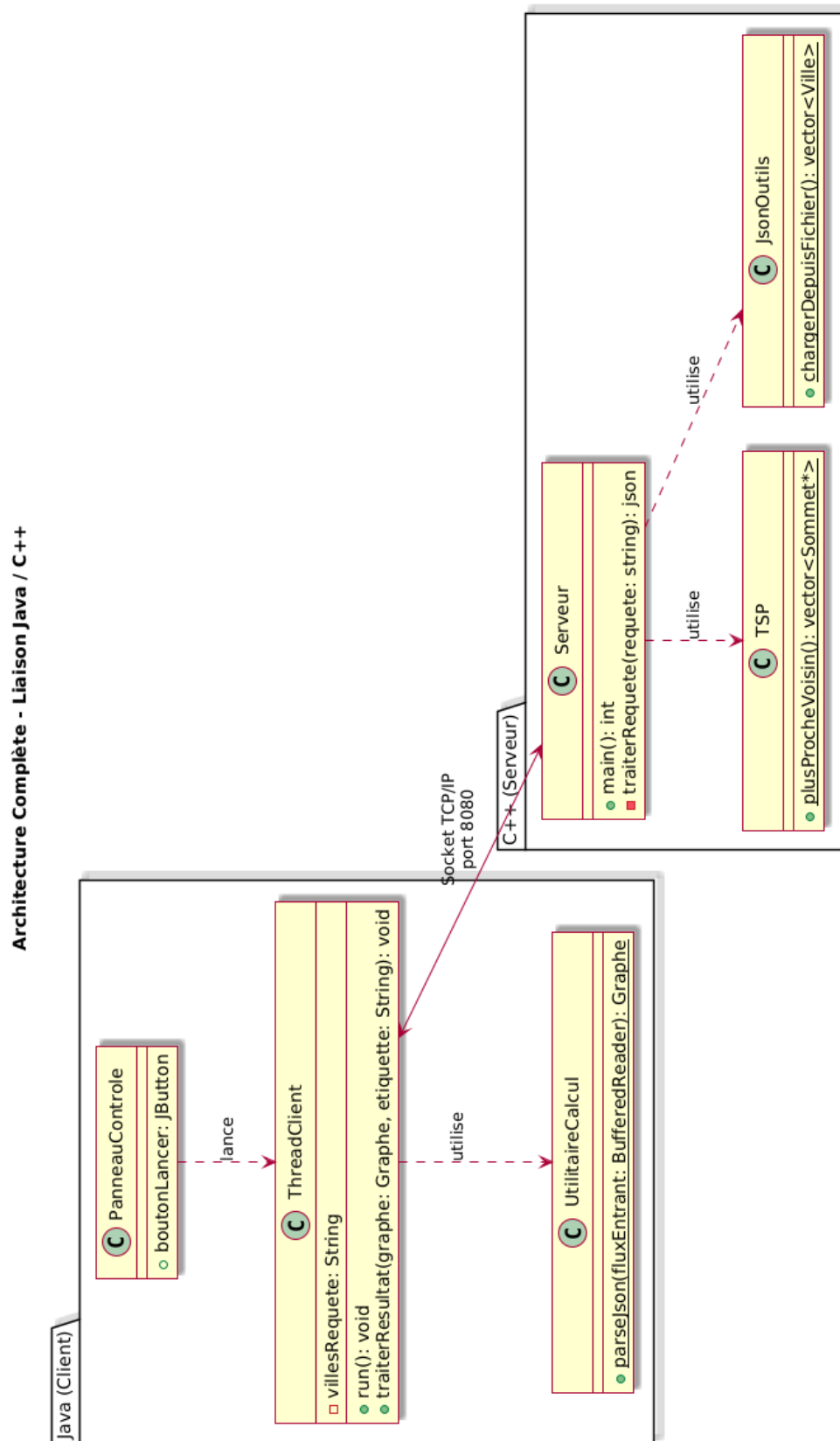


FIGURE 4 – Diagramme de Classe : Liaison java/C++

10

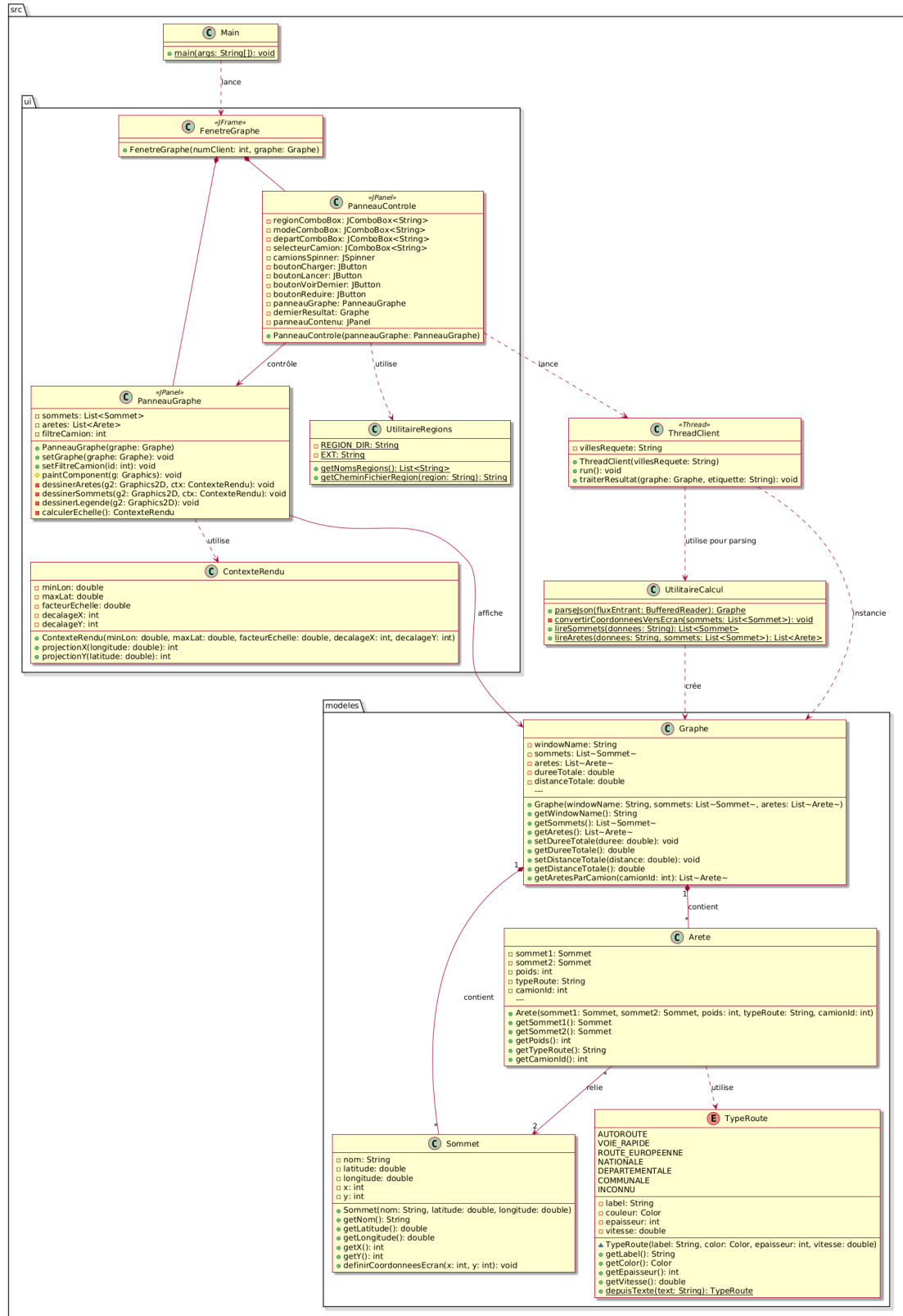


FIGURE 6 – Diagramme de Classe : Partie Java (interface / Modèles)

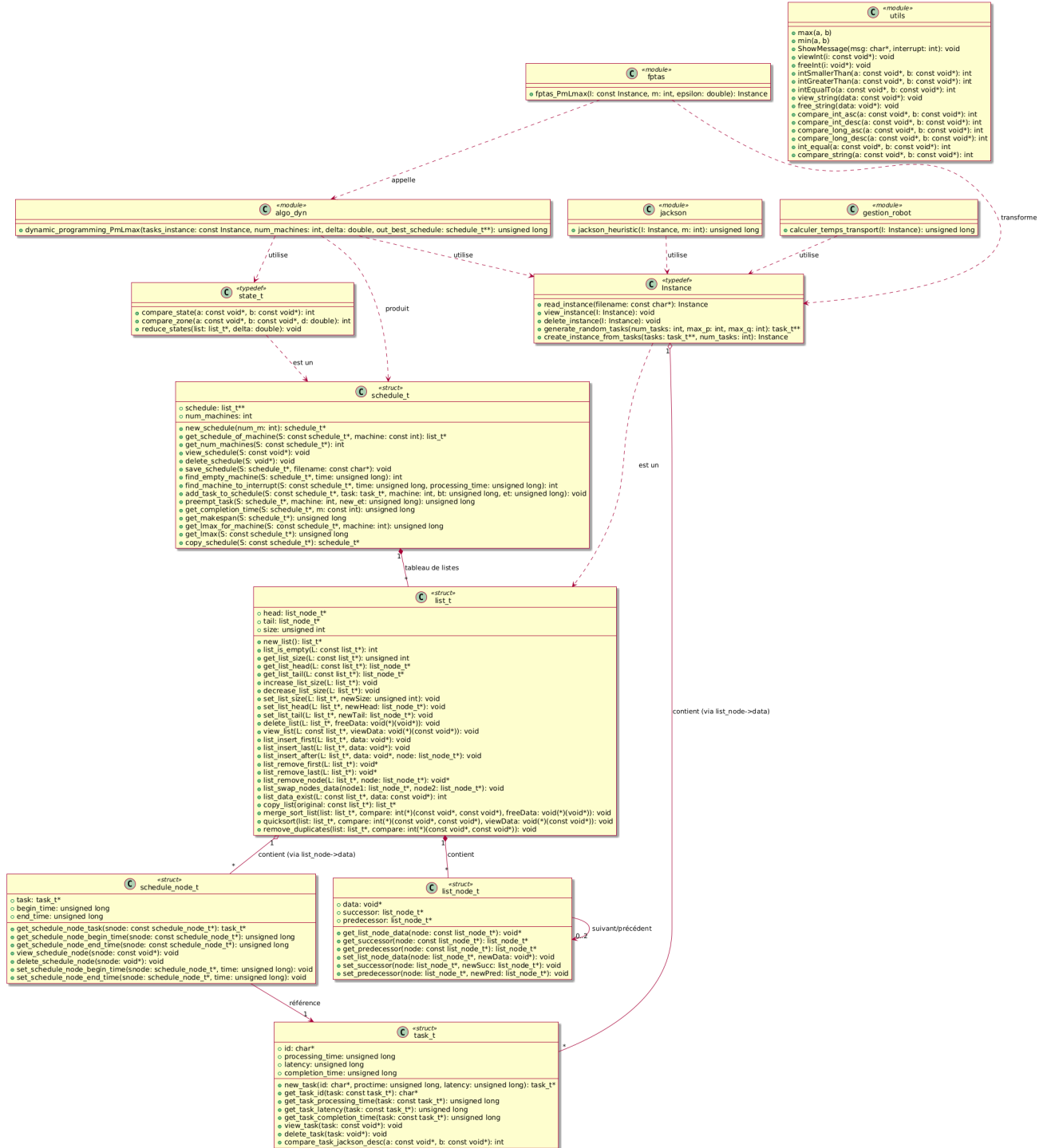


FIGURE 7 – Diagramme de Classe : Structure de données et algorithmes (C)

4 Exigences fonctionnelles (EF)

4.1 Identification des Acteurs

L'analyse du sujet a permis d'identifier un acteur principal interagissant avec le système :

L'Opérateur : Il a besoin de visualiser clairement les tournées de chaque camion sur une carte et doit connaître le calcul du chemin le plus court pour les villes assignées.

4.2 Liste des Exigences Fonctionnelles

- **EF1 : Gestion des données d'entrée.** Le système doit pouvoir importer et lire la liste d'affectations des machines et les fichiers JSON des villes nécessaires à la création du graphe.
- **EF2 : Ordonnancement (Répartition des tâches).** Le logiciel doit affecter automatiquement chaque ville (tâche) à un camion (machine) spécifique afin de préparer les tournées.
- **EF3 : Modélisation du Graphe.** Le serveur C++ doit construire un graphe complet où les sommets sont les villes et les arêtes sont les routes rectilignes.
- **EF4 : Calcul de distance.** Le serveur C++ doit calculer la distance géodésique entre deux villes pour définir le poids des arêtes du graphe.
- **EF5 : Résolution du TSP.** Le serveur C++ doit implémenter une heuristique pour trouver le chemin le plus court (TSP) pour chaque tournée de camion en utilisant un algorithme d'approximation.
- **EF6 : Configuration des variables.** L'utilisateur doit pouvoir configurer les variables du problème, telles que le choix du dépôt de départ ou le nombre de camions disponibles dans la flotte.
- **EF7 : Visualisation graphique dynamique.** Le client doit afficher une carte où les tournées de chaque véhicule sont dessinées de manière distincte.

5 Exigences non-fonctionnelles (ENF)

- **ENF1 :** Le système doit impérativement séparer les calculs (Serveur C++) de l'affichage (Client Java) en communiquant via des sockets.
- **ENF2 : Qualité du Code (Réutilisabilité).** Le code doit être conçu en exploitant au maximum les templates et les design patterns pour garantir la réutilisabilité et la maintenabilité.
- **ENF3 : Performance.** Le temps de réponse du serveur pour le calcul du TSP doit être adéquat (inférieur à quelques secondes) pour un nombre réaliste de villes.
- **ENF4 : Ergonomie et affichage.** L'interface doit résoudre correctement le problème de passage des coordonnées « monde » (latitudes/longitudes) aux coordonnées « écran » (pixels) pour une visualisation fidèle et claire.
- **ENF5 : Robustesse.** Le serveur doit intégrer des mécanismes pour vérifier la validité des entrées données par le client.

- **ENF6 : Documentation/Rigueur.** Le rapport final doit être rédigé sous forme de cahier des charges professionnel.

6 Définition des critères de qualité

6.1 CMM

Le modèle de maturité des capacités (CMM) est une méthodologie pour créer et affiner le processus de développement d'applications. Elle permet en tant que développeur d'affiner et d'améliorer le processus de développement de logiciels. Développé par le Software Engineering Institute (SEI), ce modèle définit 5 niveaux de maturité. Allant du niveau initial au niveau optimisé. Nous situons ce projet au niveau 3 (Défini). L'organisation du projet suit un processus standardisé et documenté, ce qui permet de garantir une cohérence entre les différentes phases de développement. Nous nous basons sur le savoir-faire acquis lors de projets antérieurs pour stabiliser nos méthodes. L'utilisation systématique de patrons de conception (Design Patterns) assure une qualité, mais garantit également la réutilisabilité et l'évolution future du code. L'ensemble de ces pratiques de gestion et de développement place ce projet au niveau 3 (Défini) du modèle CMM.

6.2 Détermination des standards de qualité

la qualité d'un logiciel s'articule autour de différents critères. Ils permettent de déterminer la valeur de notre application. Cette analyse se tournera à travers six critères d'évaluations de la qualité d'un logiciel :

La Convivialité : Un apprentissage aisé et une facilité d'utilisation.

L'Efficacité : l'optimisation des ressources (temps de calcul, mémoire, etc).

La Conception Modulaire (Design Patterns) : l'utilisation de patrons de conception éprouvés pour structurer le code de manière logique, découplée et standardisée.

La Fiabilité d'exécution : la capacité du logiciel à maintenir un niveau de service stable et à gérer les Difficulté.

La Facilité de maintenance : Une facilité à modifier et à faire évoluer l'application.

La Réutilisabilité : la capacité à pouvoir réutilisées des parties du code facilement.

L'analyse ci-après démontre comment nos choix de conception (architecture client-serveur, design patterns, algorithmes d'approximation) répondent à ces impératifs de qualité.

6.2.1 Convivialité

La convivialité repose sur la facilité d'utilisation. C'est pourquoi notre application est pensée pour avoir une séparation nette entre le moteur de calcul et l'interaction utilisateur. L'utilisation de différentes bibliothèques Java tel que AWT et Swing, permet d'offrir une interface graphique standard et intuitive, où l'utilisateur peut sélectionner visuellement sa région et son mode de calcul sans manipuler de lignes de commande complexes.

6.2.2 Efficacité

L'efficacité constitue un pilier central de notre application. Le langage C++ pour le serveur est dicté par le besoin de puissance de calcul pour les algorithmes du voyageur de commerce (TSP). Cela permet de traiter des graphes complexes avec un temps de réponse minimal. De plus l'optimisation en temps était un axe très important dans la réalisation du projet. L'utilisation de structures de données légères (vecteurs, listes d'adjacence) et d'algorithmes d'approximation, permet d'obtenir des solutions satisfaisantes sans saturer la mémoire vive ni le processeur.

6.2.3 La Conception Modulaire (Design Patterns)

Le projet adopte une architecture Client-Serveur : le serveur C++ assure la puissance de calcul (TSP), tandis que le client Java gère l'interface graphique. Cette structure est renforcée par plusieurs patrons de conception :

- Strategy Pattern (C++) : Utilisé pour le calcul des poids (Distance vs Temps). Grâce à l'interface `IStrategieTSP`, l'algorithme TSP reste indépendant du mode de transport choisi. Cela permet d'ajouter de nouveaux critères d'optimisation sans modifier le code existant.
- Visitor Pattern (C++) : Ce patron est utilisé pour séparer l'algorithme de traitement des données de la structure des objets eux-mêmes (Sommet, Arete). En implémentant l'interface `IVisiteur`, nous avons pu déporter la logique d'affichage (`VisiteurAffichage`) et de calcul de rapports (`VisiteurStatistiques`) hors des classes métiers. Cette approche respecte le principe de responsabilité unique (SRP) : les classes du graphe gèrent la topologie, tandis que les visiteurs gèrent les opérations spécifiques, rendant le système facilement extensible à de nouveaux types d'analyses sans modifier les classes de base.
- Context Pattern (Java) : L'objet `RenderContext` centralise les paramètres de projection (échelles, offsets). Il évite de surcharger les méthodes de dessin et découple proprement la logique de calcul géographique de la logique d'affichage AWT.
- Utility Factory Patterns : Les classes `JsonOutils` (C++) et `Utils` (Java) isolent la transformation des données brutes (JSON/CSV) en objets métiers (Ville, Sommet). Elles garantissent que les changements de formats de fichiers n'impactent pas le cœur algorithmique.
- Data Transfer Object (DTO) : Le format JSON sert de passerelle entre les deux langages. Il permet un transfert d'état structuré et léger du graphe, assurant une communication réseau fiable et indépendante des types de données spécifiques à chaque langage.

Pour le renforcement de la modularité de notre application, nous avons ajouter des foncteurs à notre architecture :

- `SelecteurMeilleurVoisin` qui permet de découpler l'algorithme du TSP de sa logique de décision et donc il devient possible de changer le critère de sélection sans modifier le code source de l'algorithme.
- `NettoyeurRequete` qui assure la fiabilité de la communication réseau en purgeant les données reçues de tout caractère parasite.

6.2.4 Stratégie de Test et Fiabilisation

Afin de garantir la robustesse du système, une attention particulière est portée à l'intégrité de la mémoire et à la fiabilité des résultats. Pour cela nous avons mis en place une campagne de tests systématiques pour chaque module majeur (Client Java et Serveur C++). Ces scénarios simulent des flux réels pour valider la chaîne de traitement complète. Nous avons intégré des tests spécifiques sur des données "atypiques", notamment la région Auvergne-Rhône-Alpes, à cause de caractère spéciaux. Ce test d'intégration a permis de détecter et de corriger un conflit d'encodage entre le format de fichier JSON et le flux de sortie du serveur. De plus grâce à ces tests d'intégrations, nous sommes passés au C++17, pour utiliser `std::filesystem`, ce qui permet au programme de gérer de manière robuste les chemins vers les fichiers de données (assets). cela permet d'éviter les erreurs de "Fichier non trouvé". Enfin une approche de **programmation défensive** permet d'anticiper les anomalies critiques. Le traitement rigoureux des exceptions, la validation stricte des flux de données et la sécurisation des accès mémoire assurent la stabilité de l'application face aux imprévus.

6.2.5 Facilité de maintenance

La maintenabilité est assurée par une architecture modulaire et bien distincte. Le projet est divisé en classes aux responsabilités uniques (JsonOutils pour le parsing, GeoOutils pour les calculs géographiques, TSP pour la logique métier). Cela permet une navigation plus aisée, et une facilité à intervenir. L'utilisation du format JSON comme protocole de communication facilite les tests. On peut tester le serveur C++ indépendamment du client Java en lui envoyant simplement des chaînes de caractères formatées.

6.2.6 Réutilisabilité

La réutilisabilité est la conséquence directe de l'insertion de nos Design Patterns : L'utilisation des patterns Utility et Factory isole les outils de parsing (JsonOutils, CsvOutils). Ces blocs logiciels peuvent être exportés vers n'importe quel autre projet sans dépendance au reste du code. De plus, grâce au Strategy Pattern, le moteur de calcul TSP et les outils de géodésie (GeoOutils) sont découplés de l'interface. Ils constituent une bibliothèque de calcul réutilisable pour toute application de logistique.

De plus, l'implémentation du Visitor Pattern renforce cette modularité : les structures du graphe (Sommet, Arete) sont totalement génériques et ne contiennent aucune logique d'affichage ou d'analyse. Cela permet de réutiliser ces classes de base dans d'autres contextes (calcul de flux, simulation de trafic) simplement en créant de nouveaux visiteurs, sans jamais modifier le code source existant.

Enfin le recours au format JSON garantit que le serveur peut communiquer avec divers clients (web, mobile, etc) sans modification, assurant une pérennité technologique.

7 Planification des étapes du développement

7.1 Jalons principaux

Pour assurer la cohérence du développement, nous avons structuré le projet autour de cinq jalons clés :

Jalons	Objectifs principaux	Livrables associés
J1 : Spécification et Conception	Analyse des besoins et modélisation logicielle.	Cahier des charges, Diagrammes UML (Classes, Séquences, États, cas d'utilisation).
J2 : Étude Algorithmique	Recherche et codage des solutions d'optimisation.	Algorithmes LPT (ordonnancement) et VRP (voyageur de commerce) en Java.
J3 : Développement et Intégration	Création de l'interface et liaison client/serveur.	Interface C++ fonctionnelle et protocole de communication avec le serveur Java.
J4 : Finalisation et Tests	Validation globale, correction des bugs.	Logiciel complet stable
J5 : Rédaction du rapport	Exposer notre conception et les choix concernant notre projet	Rapport final complet

TABLE 1 – Planification des jalons du projet

Ce découpage par jalons, détaillé dans le **tableau 1**, structure la trajectoire de développement et sécurise la livraison du logiciel. L'enchaînement de ces étapes garantit une gestion fiable, où chaque phase est conditionnée par la validation technique des étapes précédents. Une attention particulière est portée à l'interdépendance du jalon 2 (Étude Algorithmique) et du jalon 3 (Intégration), moments pivots où la cohérence entre le moteur de calcul en C++ et l'interface Java est mise à l'épreuve. Cette méthodologie de développement incrémental réduit les risques de problème sur le long terme et permet d'isoler les anomalies dès leur apparition. Enfin, les phases de test et de documentation assure la transition du prototype vers une solution logicielle industrielle, fiable et maintenable.

7.2 Diagramme de Gantt



FIGURE 8 – Diagramme de Gantt : Organisation et Répartition des tâches du projet

8 Description des exigences d'assistance technique

Lors du déploiement du logiciel, l'entreprise recevra dans un premier temps le logiciel complet, conformément à ce qui a été décidé avec le prestataire. De plus, une documentation d'exploitation sera fournie, contenant un guide d'installation technique et un manuel utilisateur.

Afin de garantir la pérennité du logiciel livré, notre équipe s'engage sur un protocole de suivi structuré.

Une première phase appelée VSR (Vérification de Service Régulier) sera mise en place sur une durée de 15 jours. Cette étape sera l'occasion pour le client de vérifier le bon fonctionnement et le respect des exigences (fonctionnelles et non fonctionnelles). L'application pourra tourner en conditions réelles. À l'issue de cette phase, si aucun bug majeur n'est constaté, le logiciel pourra être officiellement considéré comme accepté.

Un Support Technique sera mis en place par une équipe dédiée. Des niveaux de support seront organisés, allant du support utilisateur à l'expertise pour une meilleure gestion des priorités d'intervention. De plus, il assurera un délai de réaction le plus efficace possible, sous 24h pour les anomalies bloquantes.

Par ailleurs, le service technique s'engage, sans frais supplémentaires, à assurer la gestion de la dette technique et du versioning. Des correctifs pourront être proposés régulièrement afin d'assurer la sécurité des communications entre le client et le serveur.

Enfin, cette assistance technique pourra bien évidemment être ajustée si les prérogatives évoluent dans le futur.

9 Conclusion

Le présent document a permis d'établir un cadre rigoureux pour le développement d'une réponse logicielle adaptée. Il a permis de répondre aux exigences métier complexes. Ce projet atteint son objectif premier, c'est-à-dire automatiser la résolution des contraintes de livraison, optimisant ainsi la performance globale de la chaîne logistique.

L'intérêt majeur de notre démarche réside dans son architecture. En séparant strictement le moteur de calcul haute performance en C++ de l'interface utilisateur en Java, nous avons créé un système à la fois puissant et accessible pour le client. L'intégration de patrons de conception permet d'assurer la pérennité du code. Cette modularité offre à l'entreprise une liberté d'évolution importante, permettant de créer de nouvelles fonctionnalités sans déstabiliser le cœur de la solution.

À terme, le potentiel d'extension du logiciel est vaste. Plusieurs pistes de développement ont déjà été identifiées pour enrichir l'expérience utilisateur et affiner la précision des résultats :

- La mise en place d'un système de notification sur mobile, afin de transmettre les feuilles de route en temps réel aux agents de terrain.
- L'intégration de variables supplémentaires, telles que l'empreinte carbone des véhicules ou la réduction des coûts liée aux péages sur les routes.
- Le passage d'une gestion régionale à une couverture nationale intégrale.

En définitive, ce cahier des charges a constitué un réel point de repère tout au long de la conception. Il garantit que la solution finale ne se contente pas de répondre à l'une

des exigences finies, mais s'inscrit dans une vision de long terme. Ce qui offre à l'entreprise un outil fiable, évolutif et parfaitement aligné avec ses ambitions de transformation numérique.